

SZÁMÍTÓGÉP ARCHITEKTÚRÁK

Központi memória (Neumann-elvű gép)

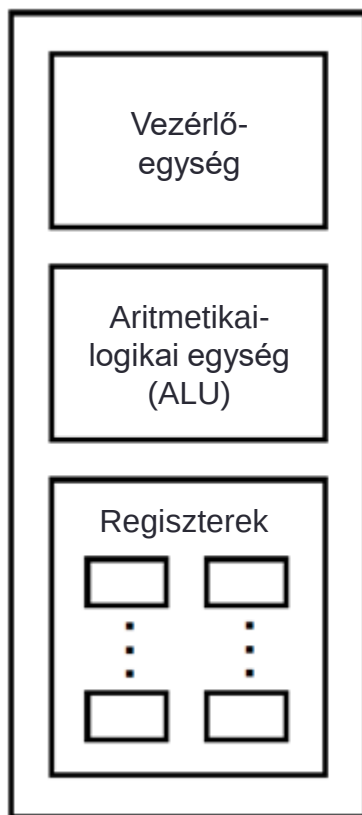
Számrendszerek, átváltások

Adatábrázolás, hibajavító kódolások

Tanács Attila

Neumann-elvű gép sematikus váza

Központi feldolgozóegység (CPU)



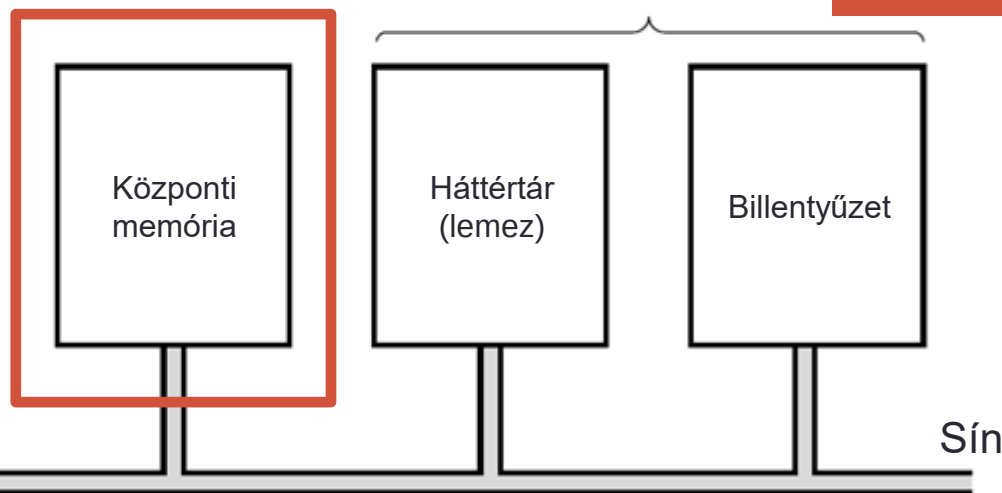
Tárolt programú digitális számítógép
alapvető komponensei, kapcsolatai

Központi memória

- Program kódja és adatai együtt
- Számokként tárolva

Harvard-elv: kód és adat külön kezelése

B/K eszközök



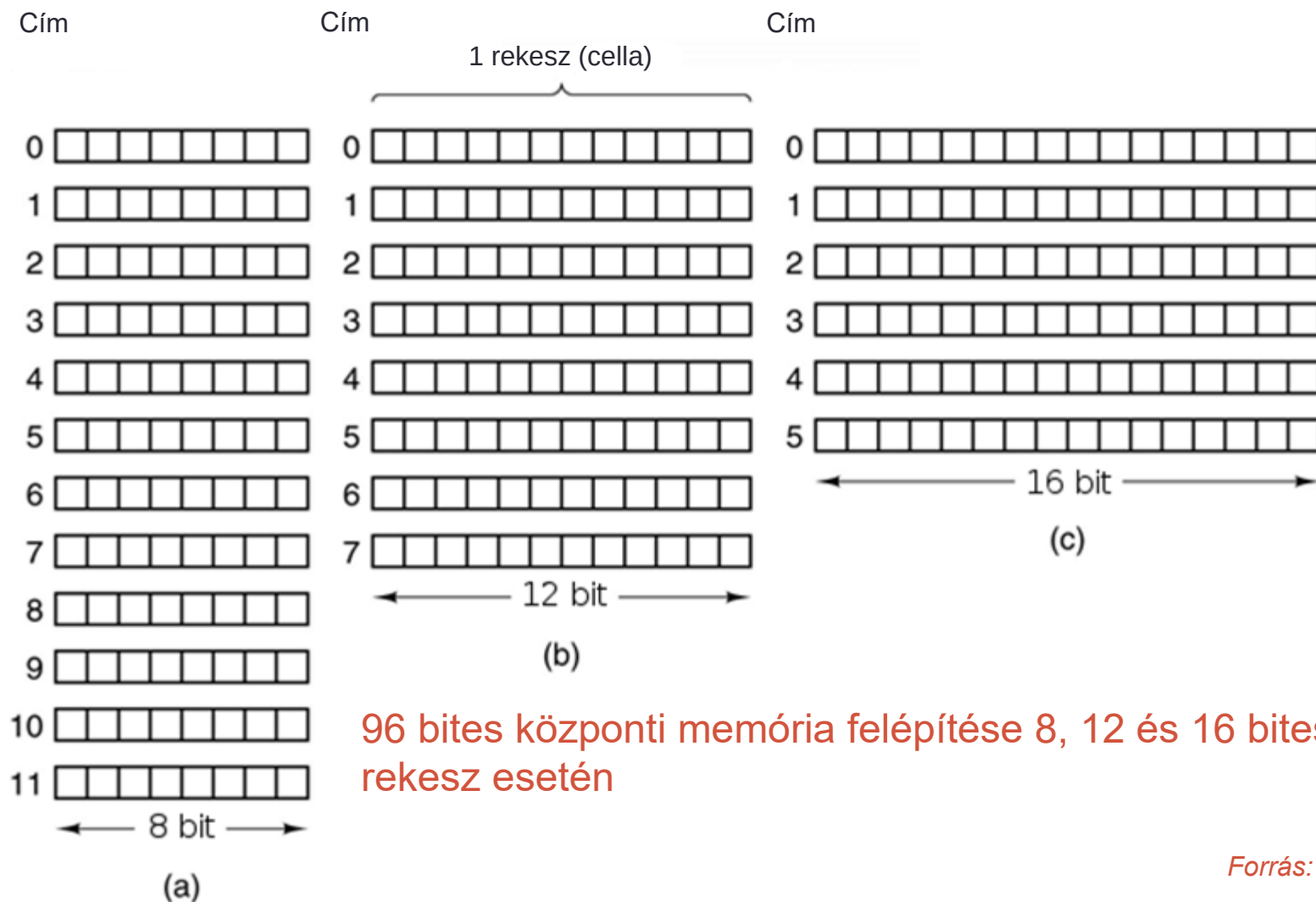
Központi memória

- Tárolás alapja a **bit**
 - Bináris 0 vagy 1 érték
 - *Van áram vagy nincs áram*
- Legkisebb címezhető egység a **rekesz (cella)**
 - Memóriában egymást követő bitek
 - Leggyakoribb: **8 bites** rekesz, a **bájt**
 - Lehetséges más csoportosítás is!
 - Újabb gépeknél a bájt a szabványos
- A memória rekeszek sorozata
- A rekeszeket a **címükkel** érhetjük el
 - Számos címezési mód!
 - ~1 dimenziós tömb
 - *Szegmens:offset* címezés

Forrás: Tanenbaum

Számítógép	Bitek/cella
Burroughs B1700	1
IBM PC	8
DEC PDP-8	12
IBM 1130	16
DEC PDP-15	18
XDS 940	24
Eletrologica X8	27
XDS Sigma 9	32
Honeywell 6180	36
CDC 3600	48
CDC Cyber	60

Központi memória



96 bites központi memória felépítése 8, 12 és 16 bites rekesz esetén

Központi memória

- **További csoportosítások**

- **Szó:** hardver által egységként kezelt bájtok
 - Sokszor a számítógép regiszterszélességének megfelelő bájtok összessége
 - 4 bájt 32-bites rendszeren, 8 bájt 64-bitesen, ...
 - Sok utasítás teljes szavakkal dolgozik
- **Paragrafus:** 16 bájt együttese, 16-tal osztható címen kezdődően
- ...

- **Elhelyezkedés a memóriában („Igazítás”)**

- Általában címezhetünk tetszőleges bájton kezdődő szavakat,
- de szóhatáron (négygyel, nyolccal, ... osztható címen) kezdődők elérése **gyorsabb lehet**.
- A fordítóprogramok általában képesek adatokat szó- vagy akár paragrafushatárra igazítani. (Viszont így kihasználatlan memóriaterületek keletkezhetnek.)

Központi memória tartalmának értelmezése

- **Például ezt találjuk a számítógép központi memóriájában**
 - 16-es számrendszerbeli számként ábrázolva, vagyis egy kétjegyű szám 1 bájt
 - A függőleges elválasztás (|) a nyolcadik szám végét jelzi
- **Mit jelenthet ez a számsor?**

```

80 07 00 05 73 75 6D 2E|63 00 88 15 00 00 00 11
42 6F 72 6C 61 6E 64 20|43 2B 2B 20 35 2E 35 2E
31 00 88 0E 00 00 E8 00|05 73 75 6D 2E 63 2E 9A
25 40 00 88 29 00 00 E9|20 28 DB 28 21 63 3A 5C
42 6F 72 6C 61 6E 64 5C|42 63 63 35 35 5C 69 6E
63 6C 75 64 65 5C 5F 6E|66 69 6C 65 2E 68 00 88
28 00 00 E9 20 28 DB 28|20 63 3A 5C 42 6F 72 6C
61 6E 64 5C 42 63 63 35|35 5C 69 6E 63 6C 75 64
65 5C 5F 6E 75 6C 6C 2E|68 00 88 28 00 00 E9 20
28 DB 28 20 63 3A 5C 42|6F 72 6C 61 6E 64 5C 42
63 63 35 35 5C 69 6E 63|6C 75 64 65 5C 5F 64 65
66 73 2E 68 00 88 2A 00|00 E9 20 28 DB 28 22 63
3A 5C 42 6F 72 6C 61 6E|64 5C 42 63 63 35 35 5C
69 6E 63 6C 75 64 65 5C|5F 73 74 64 64 65 66 2E
68 00 88 28 00 00 E9 20|28 DB 28 20 63 3A 5C 42
6F 72 6C 61 6E 64 5C 42|63 63 35 35 5C 69 6E 63
6C 75 64 65 5C 73 74 64|69 6F 2E 68 00 88 00 00
00 E9 2E 9A 25 40 05 73|75 6D 2E 63 00 88 03 00
00 E9 00 96 28 00 05 5F|54 45 58 54 04 43 4F 44
45 00 05 5F 44 41 54 41|04 44 41 54 41 06 44 47
52 4F 55 50 04 5F 42 53|53 03 42 53 53 00 98 07
00 A9 23 00 01 02 00 00|98 07 00 A9 0C 00 04 05
00 00 98 07 00 A9 00 00|07 08 00 00 9A 06 00 06
FF 03 FF 02 00 8C 17 00|08 5F 5F 73 65 74 61 72
67 76 5F 5F 00 07 5F 70|72 69 6E 74 66 00 00 90
0C 00 00 01 05 5F 6D 61|69 6E 00 00 00 00 A0 27
00 01 00 00 55 8B EC B8|05 00 00 00 BA 07 00 00
00 03 D0 8B C2 50 68 00|00 00 00 E8 00 00 00 00
83 C4 08 33 C0 5D C3 00|9D 09 00 E4 13 54 02 A4
18 56 02 00 A0 10 00 02|00 00 4F 73 73 7A 65 67
3A 20 25 64 0A 00 00 8B|02 00 00 00

```

Központi memória

- Rekeszek értékeinek értelmezése

- *Adat*

- Szám (egész, lebegőpontos, BCD, ...)
 - Karakterkód (ASCII, UTF, ...)

- *Gépi kód*

- A számok utasításokat és azok operandusait jelentik
 - A számítógép csak ezeket tudja végrehajtani!
 - Magas szintű nyelveket fordítani (vagy értelmezni) kell!

- Rekesz műveletek

- Rekesz értékének *olvasása*

- Rekesz értékének *módosítása (felülírása)*

- Lehetnek csak olvasható rekeszek is!
 - Kódterület felülírása önmódosító kódot eredményez! Veszélyes lehet!
 - A modern operációs rendszerek védik a kódterületet felülírás ellen!

Központi memória

- Mérete

- Személyi számítógépek
 - Korábban: pár kilobájt, néhány megabájt
 - Pl. A Commodore 64 gép 64 kilobájtot ért el, ez megfelel egy 256x256 méretű szürkeárnyaltos kép mátrixának!
 - Manapság: több gigabájt a jellemző
- Mikrovezérlők
 - Néhány száz vagy ezer bájt

- Felosztása

- Nagy része a programok számára szabadon felhasználható
- Bizonyos címterületek a hardverrel való kapcsolattartásra lehetnek fenntartva
 - Pl. kijelző, merevlemez, külső meghajtók
- Bizonyos címek meghatározhatják egyes címterületek tartalmát
 - Pl. RAM (írható/olvasható) vagy ROM (olvasható) legyen ott elérhető
 - Átlapolási lehetőség a címtartományban

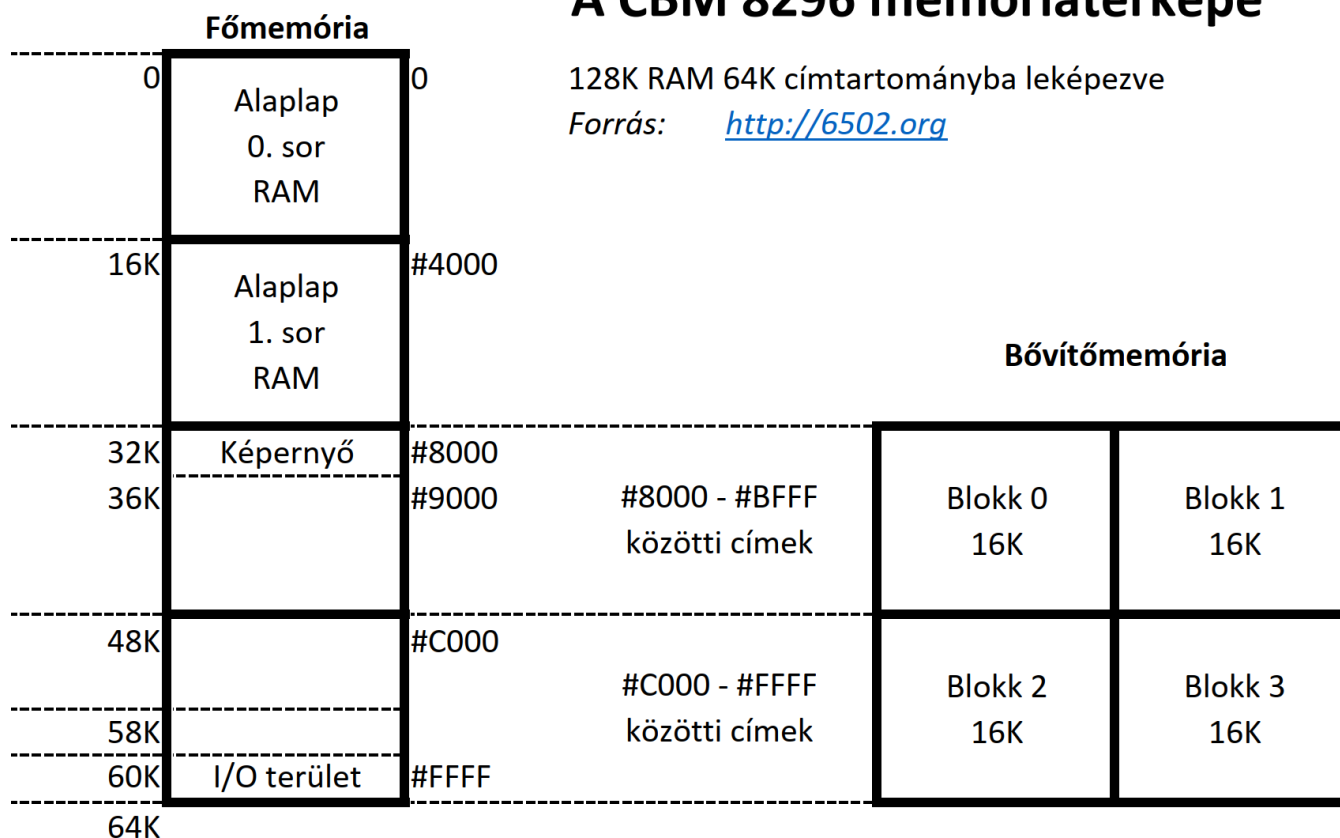
Központi memória

- Felosztása
 - Memória átlapolás példa CBM 8296 esetén

A CBM 8296 memóriatérképe

128K RAM 64K címtartományba leképezve

Forrás: <http://6502.org>



Kis kitérő

- Cél

- Számok, karakterek digitális gépi reprezentációja
 - Egész, lebegőpontos, előjel nélküli, előjeles, ...

- Eszköz

- Számrendszerek ismerete
- Véges pontosságú számok fogalma
- IEEE 754-es lebegőpontos szabvány
- ASCII, ISO-8852 és Unicode karakterkódolás

Mennyiségek számon tartása



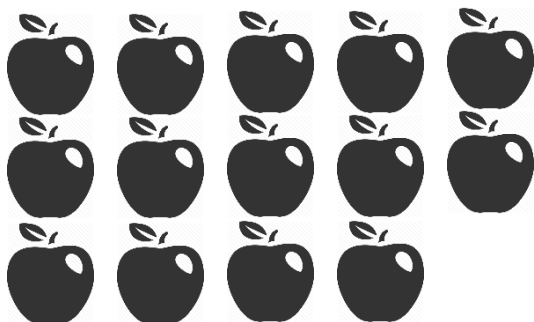
Egy darab alma



Két darab alma



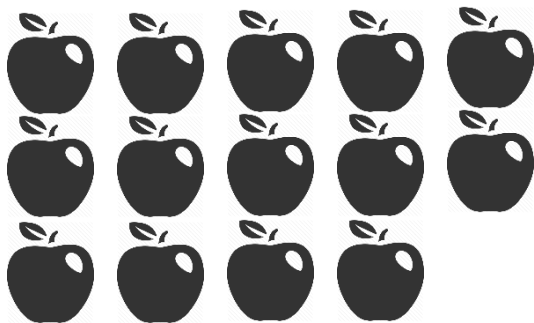
Öt darab alma



Tizennégy darab alma

Meg kell tudnom absztrakt módon nevezni a mennyiségeket, hogy kommunikálni tudjak róluk!

Absztrakció



Csoportosítás



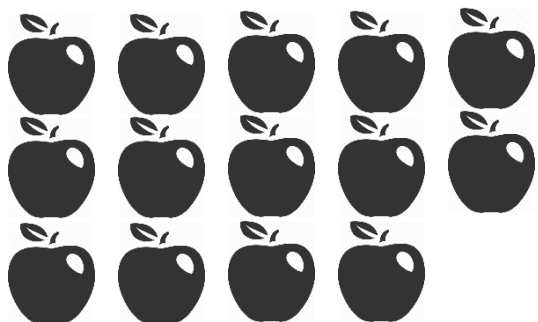
I



II



||||



|||| |||| |||

Számfogalom kialakulása



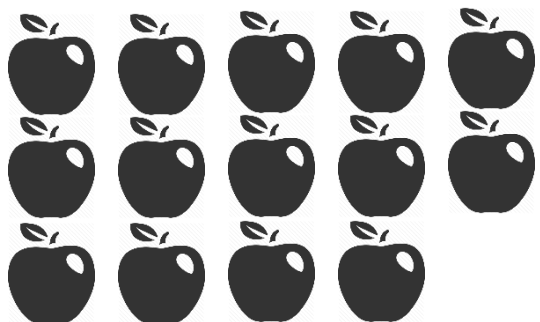
I: 1



II: 2



III: 1 áthúzott + 0 marad



III III III:

2 áthúzott + 4 marad

Szimbólumok hozzárendelése



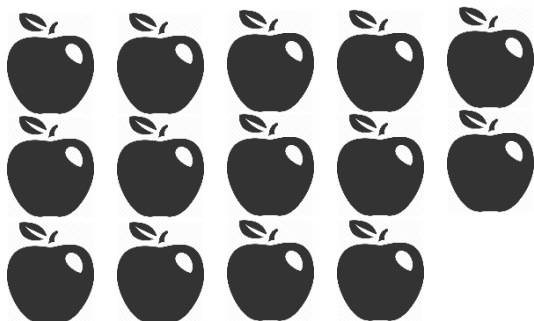
I: 1



II: 2



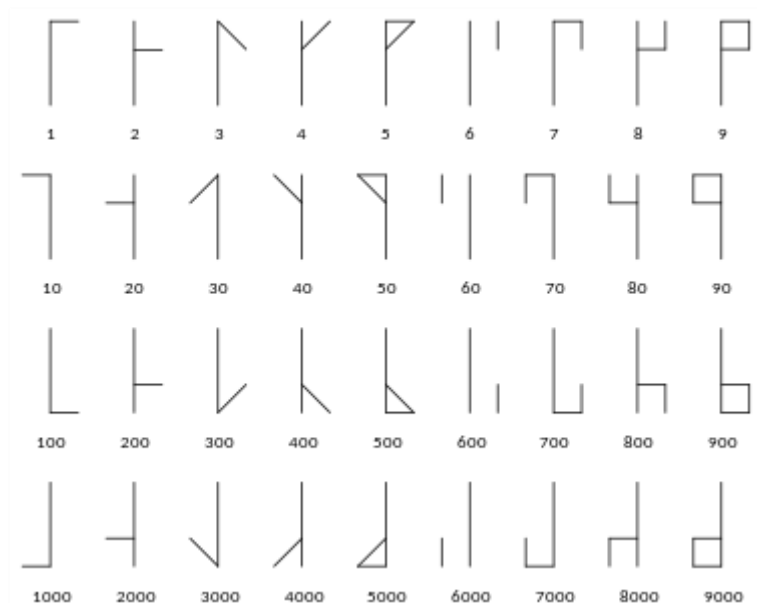
HH: 10



HH HH III: 24

Ötös számrendszer

Érdekesség



Kép szerzője: [Meteoorkip - Own work](#), CC BY-SA 4.0

• Ciszterci számok

- 1 és 9999 közötti számok leírása egy darab szimbólummal
- Angol nyelvű leírás:
 - https://en.wikipedia.org/wiki/Cistercian_numerals
- Magyar nyelvű leírás:
 - <https://index.hu/tudomany/til/2021/04/26/ciszterciek-szamrendszere/>

Számrendszerek

- **Alapja**

- Mennyiségi egységek, csoportosítás, helyiértékes ábrázolás

- **Népszerű számrendszerek**

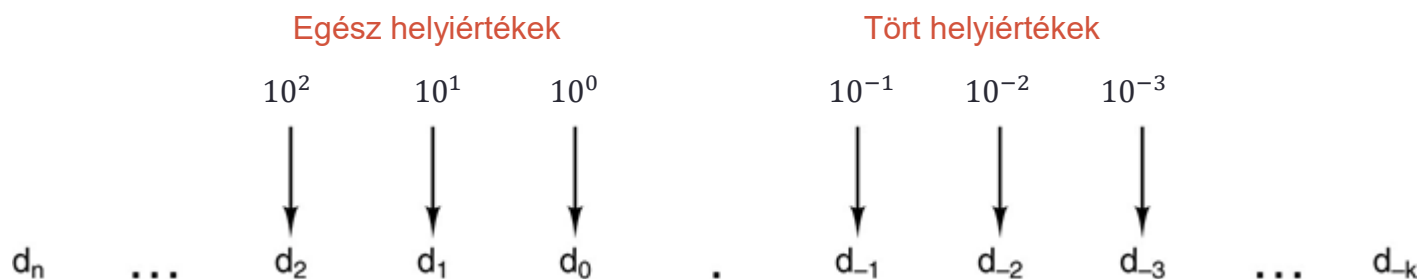
- 10-es: mindennapi életben, gazdaság-üzlet
 - 10 ujjunk van (?)
- 2-es (bináris): digitális számítógépek alaprendszere
 - 8-as (oktális), 16-os (hexadecimális): bináris számok tömörebb felírási lehetősége

- **Más mennyiségi egységek**

- A sumér 10-12-60-as számrendszer örökségei
 - Óra/perc: 12/60 (I-XII)
 - Szög, perc, másodperc: 360/60/60
- ...

Számrendszerek

- **Tízes számrendszer (*decimális*)**
 - Hétköznapi életben ezt használjuk leggyakrabban
 - 0-9 számjegyek (10 darab)
 - Hatványkifejezés alapszáma (*radix*) a 10



$$\text{számérték} = \sum_{i=-k}^n d_i \cdot 10^i$$

$$1 \cdot 100 + 5 \cdot 10 + 2 \cdot 1 + 3 \cdot 0,1 + 9 \cdot 0,01 + 6 \cdot 0,001$$

Például:

1 5 2 . 3 9 6

Számrendszerek

Két informatikus barkochbázis:

- Gondoltam egy számra!
- Egy?
- Nem.
- Akkor nulla!

- **Kettes számrendszer (*bináris*)**

- Kétféle számjegy: **0** és **1** (radix: 2)
- Megfeleltethető a biteknek, emiatt a digitális számítógép tkp. ezt használja

- **Tizenhatos számrendszer (*hexadecimális*)**

- 16-féle számjegy: **0-9** és **A-F** között (radix: 16)
- Egy 4 számjegyű bináris szám egy darab 16-osbelin ábrázolható
- **Bináris tömörebb leírási módja!**

- **Nyolcas számrendszer (*oktális*)**

- Nyolcféle számjegy: **0-7** között (radix: 8)
- Egy 3 számjegyű bináris szám egy darab nyolcas számrendszerbelivel leírható

$$d_n \dots d_2 \ d_1 \ d_0 \ . \ d_{-1} \ d_{-2} \dots d_{-k}$$

$$decimális_számérték = \sum_{i=-k}^n d_i \cdot radix^i$$

Számrendszerek példa

Bináris	1	1	1	1	1	0	1	0	0	0	1											
	$1 \times 2^{10} + 1 \times 2^9 + 1 \times 2^8 + 1 \times 2^7 + 1 \times 2^6 + 0 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$																					
	1024	+	512	+	256	+	128	+	64	+	0	+	16	+	0	+	0	+	0	+	0	+
Oktális	3	7	2	1																		
	$3 \times 8^3 + 7 \times 8^2 + 2 \times 8^1 + 1 \times 8^0$																					
	1536	+	448	+	16	+	1															
Decimális	2	0	0	1																		
	$2 \times 10^3 + 0 \times 10^2 + 0 \times 10^1 + 1 \times 10^0$																					
	2000	+	0	+	0	+	1															
Hexadecimális	7	D	1	.																		
	$7 \times 16^2 + 13 \times 16^1 + 1 \times 16^0$																					
	1792	+	208	+	1																	

A 2001 egész szám többféle számrendszerben kifejezve

Számrendszerek példa

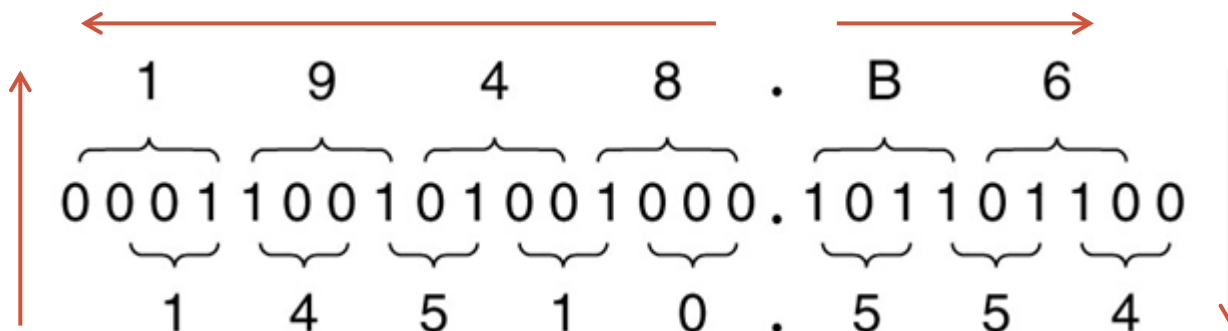
Szám	Bináris	Oktális	Hexa
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E

15	1111	17	F
16	10000	20	10
20	10100	24	14
30	11110	36	1E
40	101000	50	28
50	110010	62	32
60	111100	74	3C
70	1000110	106	46
80	1010000	120	50
90	1011010	132	5A
100	11001000	144	64
1000	1111101000	1750	3E8
2989	101110101101	5655	BAD

Átváltások számrendszerek között

• Bináris-oktális, bináris-hexadecimális átváltás

- Bináris számjegyek négyes csoportja 1 hexadecimális számjegy
- Bináris számjegyek hármass csoportja 1 oktális számjegy
- Csoportosítás a tizedesponttól indul
 - Egész értékeknél balra, tört értékeknél jobbra
 - Vezető és záró 0 számjegyek szükségesek lehetnek

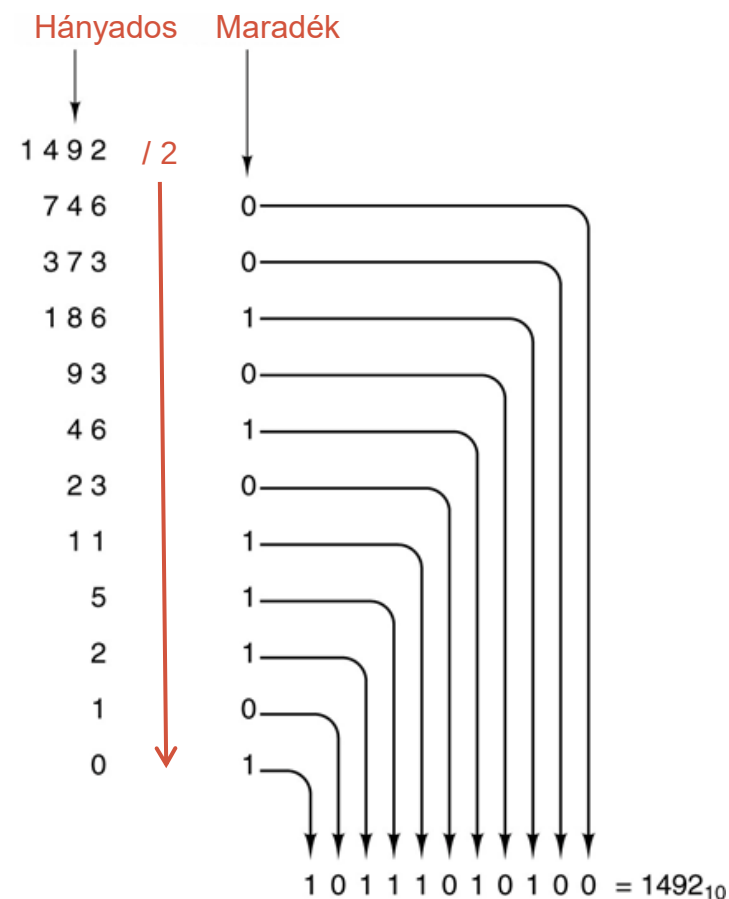


• Oktális-hexadecimális átváltás

- Pl. a bináris számrendszer felhasználásával: oktális $\leftrightarrow$ bináris $\leftrightarrow$ hexa

Átváltások számrendszerek között

- **Egész érték decimálisból binárisba**
 - A decimális értéket osztjuk 2-vel
 - Feljegyezzük
 - a hányados egész részét
 - és a maradékot
 - Ismételjük, míg a hányados 0 nem lesz
 - A maradék értékek adják a bináris számjegyeket
 - Az első osztás a legkisebb helyiértékű számjegyet adja
- **Átváltás más számrendszerbe**
 - A számrendszer alapszámával végezzük az osztást
- **Feladat**
 - Más számokra is hajtsuk végre az átváltó algoritmust!
 - Más számrendszerbe is váltsunk át így!



Átváltások számrendszerek között

- (0, 1) közötti tört érték decimálisból binárisba
 1. A decimális értéket szorozzuk 2-vel
 2. Ha 1, akkor 1-es számjegyet írunk és kész
 3. Ha 1-nél nagyobb, akkor 1-es számjegyet írunk, kivonunk 1-et, és folytatjuk az 1. lépéssel
 4. Ha 1-nél kisebb, akkor 0-s számjegyet írunk és folytatjuk az 1. lépéssel
 - Az első szorzás a legnagyobb helyiértékű (2^{-1}) számjegyet adja
 - Utána egyesével lépkedünk jobbra, a kisebb helyiértékek felé
 - Nem biztos, hogy véges lesz!

Átváltások számrendszerek között

- Tört átváltás decimálisból binárisba (példák)

Helyiérték	0,3125 (10) = 0,0101 (2)	0,26 (10) = 0,0100001...
$2^{-1} = 0,5$	$0,625 = 2 * 0,3125 \rightarrow \mathbf{0}$	$0,52 = 2 * 0,26 \rightarrow \mathbf{0}$
$2^{-2} = 0,25$	$0,25 = 2 * 0,625 - 1 = 1,25 - 1 \rightarrow \mathbf{1}$	$0,04 = 2 * 0,52 - 1 = 1,04 - 1 \rightarrow \mathbf{1}$
$2^{-3} = 0,125$	$0,5 = 2 * 0,25 \rightarrow \mathbf{0}$	$0,08 \rightarrow \mathbf{0}$
$2^{-4} = 0,0625$	$0 = 2 * 0,5 - 1 = 1 - 1 \rightarrow \mathbf{1}$	$0,16 \rightarrow \mathbf{0}$
$2^{-5} = 0,03125$		$0,32 \rightarrow \mathbf{0}$
$2^{-6} = 0,015625$		$0,64 \rightarrow \mathbf{0}$
$2^{-7} = 0,0078125$		$0,28 = 1,28 - 1 \rightarrow \mathbf{1}$
...	Véges!	... Végtelen ismétlődés (0,52 jön ki újra majd)

- Feladat

<https://binary2hex.com/numberconverter.html>

- Írjunk fel további törtszámokat és alakítsuk át bináris számrendszerbe őket!

Átváltások számrendszerek között

• Binárisból decimálisba

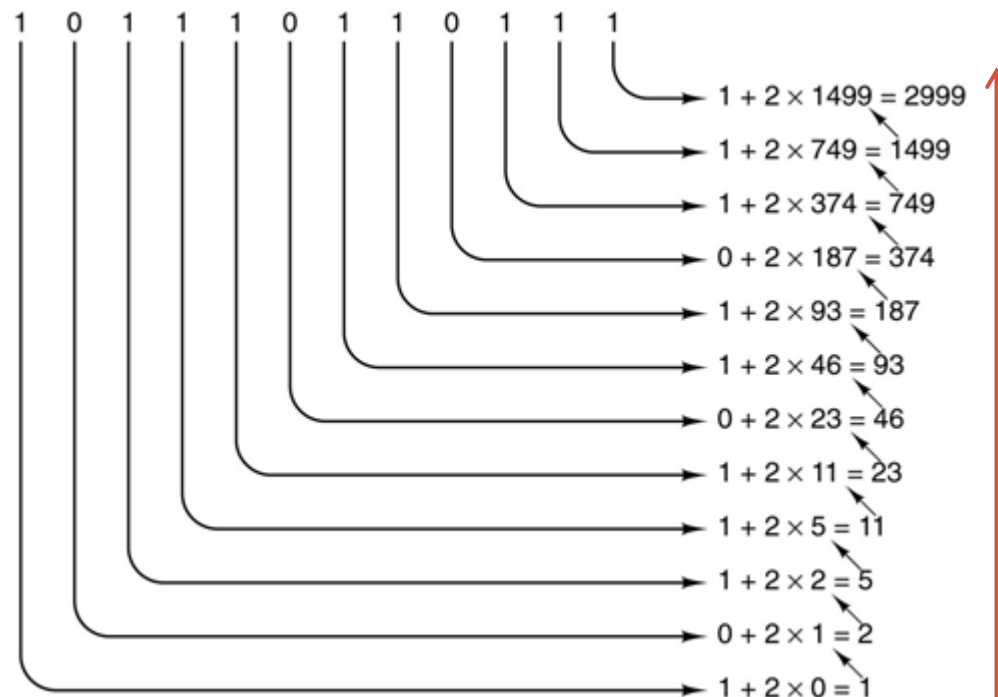
- A $\sum_{-k}^n d_i \cdot 2^i$ számítási mód sok felesleges számítást tartalmaz
- A hatványértékek számítása összevonható
 - Optimális: Horner-elrendezés

Forrás: Tanenbaum

Példán keresztül:

101110110111 bináris szám
átváltása decimálisba (2999)

1. Legmagasabb helyiértékű bináris számjegytől indulunk
2. megszorozzuk 2-vel és hozzáadjuk a következő bináris számjegy értékét
3. A 2. lépést ismételjük, míg vannak számjegyek



Véges pontosságú számábrázolás

- Digitális ábrázolás

- A központi memória véges számú rekeszből (bájtból) áll
- A hatékony munkához: tárolás rögzített számú biten/bájton
 - Csak adott értéktartományt tudnak reprezentálni
 - Számok véges pontossággal ábrázolhatók
 - Az ábrázolható tartomány minden eleme egyforma méretű memóriaterületen tárolódik

- Véges pontosságú számok jellemzői

- Túlcsordulás
 - művelet eredménye nagyobb a legnagyobb ábrázolhatónál
- Alulcsordulás
 - művelet eredménye kisebb a legkisebb ábrázolhatónál
- Példa: három számjeggyel ábrázolható egész számok: 000, ..., 999
 - $600 + 600 = 1200$ (nem ábrázolható, túl nagy) → lehet pl. 999 vagy 200
 - $003 - 005 = -2$ (nem ábrázolható, túl kicsi) → lehet pl. 000 vagy 998
 - $007 / 002 = 3.5$ (nem ábrázolható, nem egész) → lehet pl. 003
 - Például az $a + (b - c)$ nem lesz egyenlő $(a + b) - c$ értékével, ha pl. $a=700$, $b=400$, $c=300$ és az alul- és túlcsordulást nem kezeljük

Egész számok ábrázolása

- **Előjel nélküli (nem negatív) egész számok**
 - Bitek helyiértékének megfelelően
 - 1 bájtos tárolás: 0 - 255 (2^8) közötti értékek
 - 2 bájtos tárolás: 0 - 65,535 (2^{16}) közötti értékek
 - 4 bájtos tárolás: 0 - 4,294,967,295 (2^{32}) közötti értékek
- **Binárisan kódolt decimális (BCD)**
 - Régi számológépekben, pénztárgépekben (és Intel processzorokon)
 - 4 (pakolt) vagy 8 bit (pakolatlan) egy decimális számjegyet jelöl
 - 100 darab (4-bites reprezentáció) vagy 10 darab (8-bites) különböző szám ábrázolható így
 - Egyes bájt értékek nem hordoznak jelentést (pl. 1011, pazarló)
 - Példák
 - 19 pakolt BCD kódolással: 0001 1001 = 19H (decimális 25)
 - 73 pakolt BCD kódolással: 0111 0011 = 73H (decimális 115)
 - 199 pakolt BCD kódolással: 0000 0001 1001 1001 = 199H (dec. 409)

Előjeles egész számok

- Előjeles abszolút érték

- Legfelső bit: előjel (0 nem negatív, 1 negatív)
- További bitek: szám abszolút értéke helyiérték szerint
- Probléma: létezik +0 és -0 is!
- Példák
 - 00000001: +1; 00000000: +0; 10000000: -0; 11111111: -127;
01111111: +127

- Egyes komplement

- Legfelső bit: előjel (0 nem negatív, 1 negatív)
- További bitek: szám értéke, de negatív esetben mindegyik ellentett értékkel (0 és 1 számjegyeket felcseréljük)
- +0 és -0 probléma itt is fennáll!
 - 00000001: +1; 11111110: -1; 00000000: +0; 11111111: -0;
01111111: +127

Előjeles egész számok

- Kettes komplementum

- Képzése: egyes komplementum felírás + 1 hozzáadása
 - Ha átvitel jelenik meg a bal szélső bitnél, azt eldobjuk
- Csak egyféle 0 érték reprezentáció! (Ez végre jó!)
- Nem szimmetrikus ábrázolási tartomány: -128 és +127 között

- 2^{m-1} többletes

- m bites szám esetén
 - $m = 8$ esetén 128 többletes
- A többletes értékét hozzáadjuk az ábrázolandó számhoz
 - Negatív számok: kisebbek a többletesnél
 - 0 értéke: többletes értéke
 - Pozitív számok: nagyobbak a többletesnél
- Egyező a kettes komplementummal, csak fordított előjelbittel
- Használata: lebegőpontos számok kitevő részénél

Számábrázolás

Decimális számok különféle bitminta reprezentációi

N decimális	N bináris	-N előjeles absz.ért.	-N előjeles 1-es komplement	-N előjeles 2-es komplement	-N 128 többletes
1	00000001	10000001	11111110	11111111	01111111
2	00000010	10000010	11111101	11111110	01111110
3	00000011	10000011	11111100	11111101	01111101
4	00000100	10000100	11111011	11111100	01111100
5	00000101	10000101	11111010	11111011	01111011
6	00000110	10000110	11111001	11111010	01111010
7	00000111	10000111	11111000	11111001	01111001
8	00001000	10001000	11110111	11111000	01111000
9	00001001	10001001	11110110	11110111	01110111
10	00001010	10001010	11110101	11110110	01110110

Számábrázolás

Decimális számok különféle reprezentációi

N decimális	N bináris	-N előjeles absz.ért.	-N előjeles 1-es komplement	-N előjeles 2-es komplement	-N 128 többletes
20	00010100	10010100	11101011	11101100	01101100
30	00011110	10011110	11100001	11100010	01100010
40	00101000	10101000	11010111	11011000	01011000
50	00110010	10110010	11001101	11001110	01001110
60	00111100	10111100	11000011	11000100	01000100
70	01000110	11000110	10111001	10111010	00111010
80	01010000	11010000	10101111	10110000	00110000
90	01011010	11011010	10100101	10100110	00100110
100	01100100	11100100	10011011	10011100	00011100
127	01111111	11111111	10000000	10000001	00000001
128	Nem ábrázolható	Nem ábrázolható	Nem ábrázolható	10000000	00000000

Forrás: Tanenbaum

Számábrázolás

- Fordított irányban is feltehető a kérdés
 - Egy bitsorozat milyen decimális értéknek felel meg?
 - Pl. 10010011
 - Attól függ, hogy az adott gépi utasítás milyen reprezentációnak tekinti

Bináris aritmetika

- Bináris számok összeadása
 - Decimális számrendszerbeli összeadással egyező módon

Első tag	0	0	1	1
Második tag	0	1	0	1
Összeg	0	1	1	0
Átvitel	0	0	0	1

Példa

- Jobbról balra haladunk
- Átvitelt figyelembe vesszük
- Utolsó átvitel kezelése
 - 1-es komplementnél hozzáadjuk a jobb szélső bithez
 - 2-es komplementnél eldobjuk, vagy tovább tudjuk görgetni nagyobb helyiértéket jelentő bájtok felé

$$\begin{array}{r}
 10 \\
 + (-3) \\
 \hline
 +7
 \end{array}$$

1-es komplement

$$\begin{array}{r}
 00001010 \\
 11111100 \\
 \hline
 \end{array}$$

$$1 \quad 00000110$$

átvitel hozzáadása jobbról

$$00000111$$

2-es komplement

$$\begin{array}{r}
 00001010 \\
 11111101 \\
 \hline
 \end{array}$$

$$1 \quad 00000111$$

átvitel eldobása

Számábrázolás

- **Bináris számok összeadási tulajdonságai**
 - Ha a két tag ...
 - ... ellentétes előjelű, akkor nem lesz túlcsordulás.
 - ... egyező előjelű, de az eredmény más előjelű, akkor túlcsordulás következett be és az eredmény rossz!
 - Legtöbb esetben megtartásra kerül az előjelbitnél bekövetkezett átvitel és a túlcsordulás
 - Pl. speciális flag regiszter bitekben
 - Így kezelhető az átviteli/túlcsordulási probléma
 - Jellemzően csak alacsony szinten (gépi kód, Assembly) érhető el, magasszintű programozási nyelvekben nem!
- **Ezek alapján a választott reprezentáció előjeles egész számokra**
 - Az évtizedek alatt kidolgozott reprezentációk közül az 1980-as évek óta *de-facto* szabvány a **kettes komplement** ábrázolás
 - Nincs ± 0 redundancia, valamint a bináris aritmetikával a számítások egyszerűen elvégezhetők
 - Ez lehetővé teszi az adatok bináris, konverzió nélküli átvitelét különféle architektúrák között

Nem egész számábrázolás

- **Fixpontos**

- Megadott számú biten az egész és tört reprezentáció
 - Pl. **1100110110**.**011011**
- Kis számábrázolási tartomány
- Pontatlan

- **Lebegőpontos**

- Törtrész és kitevő segítségével
- IEEE 754 szabvány: manapság általánosan használt

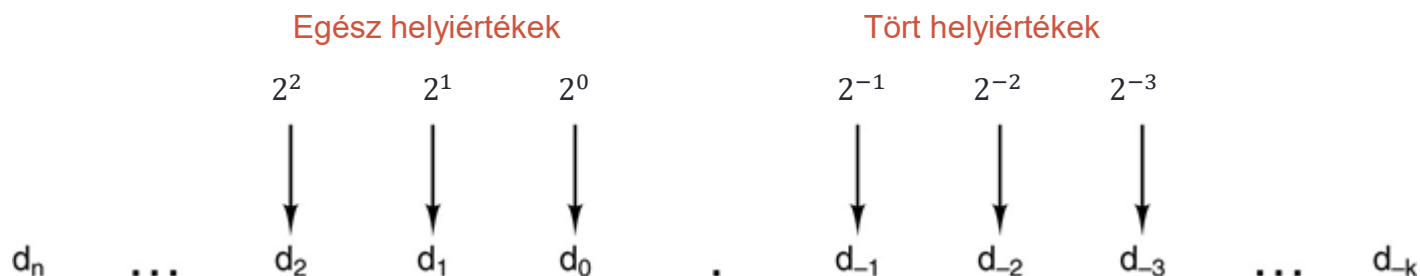
Fixpontos számábrázolás

- Rögzített méret az egész- és törtrésznek
 - Melyik hány bitet „kap”
 - Egész aritmetikával dolgozhatunk
- A mai számítógépek jellemzően nem használják
 - Kis számítási kapacitású, lebegőpontos számításokra hardveresen nem képes rendszerekben opció (beágyazott, mikrovezérlők, stb.)

Fixpontos számábrázolás

• Példák 32 bites esetekre

- 1 bit előjel, 23 bit egészrész, 8 bit törtrész
 - Az egészrész abszolút nagyságrendje 2^{23} (~8,4 millió), a törtrészé 2^{-8} (~0,0039). Pontatlan a törtrész ábrázolása!
- Másik lehetőség: 1 bit előjel, 16 bit egészrész, 15 bit törtrész
 - Kisebb az egészrész tartománya ($2^{16} = 65,5$ ezer), de valamennyit javul a törtrész ($2^{-15} \sim 0.00003$). Gazdasági számításokhoz továbbra is alkalmatlan!



$$\text{számérték} = \sum_{i=-k}^n d_i \cdot 2^i$$

Lebegőpontos számok

- $számérték = f \cdot radix^e$
 - f : törtrész (*fraction*) vagy mantissza
 - e : kitevő vagy exponens
 - $radix$: alkalmazott számrendszer
 - Számítógépen jellemzően 2-essel dolgoznak 10-es helyett!
- **Többféle reprezentáció lehetséges, ami ugyanazt jelenti**
 - A kitevő balra-jobbra tolja a tizedes/binárisvesszőt
 - Tízes számrendszerbeli példák:
 - $3,14 = 0,314 \cdot 10^1 = 3,14 \cdot 10^0$
 - $0,000001 = 0,1 \cdot 10^{-5} = 1,0 \cdot 10^{-6} = 10,0 \cdot 10^{-7}$

Jellemzők

- **Nagyságrend**
 - Kitevő jegyeinek száma határozza meg
- **Pontosság**
 - Törtrész jegyeinek száma határozza meg
- **A valós számhalmaz csak egy véges részét lehet reprezentálni!**
 - Mivel két valós szám között végtelen sok további valós szám található
 - Összehasonlító műveletek esetén a különbségük nagyságát nézzük a bináris minta összehasonlítása helyett!
 - Ha kisebb egy megadott értéknél, akkor egyenlőnek vesszük
- **Az ábrázolási pontosság nem egyenletes**
 - Pl. nagyobb abszolútértékű számok esetén pontatlanabb

Normalizálás

- **Megegyezés alapján lévő helyen** – pl. a tizedesvessző mellett jobbra (vagy balra) – **legyen az első nem 0 számjegy**
- **Példák** (2-es alapú hatvány alkalmazásával, 24 biten tárolva)
 - $432 = 0 \mid 1010100 \mid 00000000000011011$ (előjel | 64-többletes kitevő:
 $84-64 = 20 \mid$ normalizálatlan törtrész $\rightarrow 2^{20} \cdot (2^{-12} + 2^{-13} + 2^{-15} + 2^{-16}) = 432$)
 - $432 = 0 \mid 1001001 \mid 110110000000000000$ (előjel | 64-többletes kitevő:
 $73-64 = 9 \mid$ normalizált törtrész $\rightarrow 2^9 \cdot (2^{-1} + 2^{-2} + 2^{-4} + 2^{-5}) = 432$)
 - Normalizáláskor 11 hellyel balra toltuk a törtrészt, amit a **kitevőnél kompenzáltunk**
- **Vegyük észre**
 - Kettes számrendszer esetén egy normalizált szám tizedesszimbólum melletti értéke biztosan **1** $\rightarrow$ felesleges tárolni (de más számrendszernél nem ússzuk meg)
- **Sokféleképpen megoldható: egységesíteni kell $\rightarrow$ IEEE 754 szabvány**
 - Hány bitek legyenek az egyes mezők?
 - Kitevőben az egész számérték hogyan legyen reprezentálva?
 - Milyen normalizálás legyen?

IEEE 754-es lebegőpontos szabvány (1985)

- **Kettes hatványalapot használ**
- **Egyszeres pontosság (32 biten)** `float` (C, Java, ...)
 - 1 előjelbit, 8 bites kitevőrész (127-többszörös ábrázolással), 23 bites törtrész
 - Decimális kiterjedés: kb. 10^{-38} -tól 10^{38} -ig
- **Dupla pontosság (64 biten)** `double` (C, Java, ...)
 - 1 előjelbit, 11 bites kitevőrész (1023-többszörös ábrázolással), 52 bites törtrész
 - Decimális kiterjedés: kb. 10^{-308} -tól 10^{308} -ig
- **Kiterjesztett pontosság (80 bit)**
 - Lebegőpontos aritmetikai egység belsejében a kerekítési hibák csökkentésére

Lebegőpontos numerikus típusok

- Normalizált

±	$0 < \text{kitevőrsz} < \text{max}$	Bitminta
---	-------------------------------------	----------

- Nem normalizált

±	0	Nem 0 bitminta
---	---	----------------

- Nulla

±	0	0
---	---	---

- Végtelen

±	111...1	0
---	---------	---

- Nem szám

±	111...1	Nem 0 bitminta
---	---------	----------------

Előjel

Kitevőrsz

Szignifikáns (~tötrész)

Egyszeres pontosság

• Normalizált számok

- 1 előjelbit, 8 bites kitevőrész (127 többletes), 23 bites törtrész
- Ha $0 < \text{kitevőrész} < 255$, akkor a szám *normalizált*
- A *normalizált* szám első jegye (tizedesvesszőtől balra) **1**, így az nincs ábrázolva!
 - A törtrészt itt *szignifikáns*nak hívjuk
- Ábrázolt szám: $\pm(1 + \text{szignifikáns}) \cdot 2^{\text{kitevő}}$,
ahol $\text{kitevő} = \text{kitevőrész} - 127$
- Példák

+1.0:	0011 1111 1000 ... 0000 ₂	= 3F80 0000 ₁₆
+0.5:	0011 1111 0000 ... 0000 ₂	= 3F00 0000 ₁₆
-1.5:	1011 1111 1100 ... 0000 ₂	= BFC0 0000 ₁₆

$\pm$
 kitevő-
 rész

1.
 szigni-
 fikáns

Egyszeres pontosság

- További példák

+15.0: 0100 0001 0111 ... 0000₂ = 4170 0000₁₆

- Előjel: nem negatív
- Kitevő: 130 – 127 = 3
- Szignifikáns: 1 + 0,5 + 0,25 + 0,125 = 1,875
- Eredmény: 1,875 · 2³ = 15

-15315.0: 1100 0110 0110 1111 0100 1100 0000 0000

- Előjel: negatív
- Kitevő: 140 – 127 = 13
- Szignifikáns: 1 + 0,5 + 0,25 + 0,0625 + 0,03125 + 0,015625 + 0,0078125 + 0,001953125 + 0,000244140625 + 0,0001220703125
- Eredmény: -1,8695068359375 · 2¹³ = -15315

Nem mindig tökéletes a reprezentáció! Pl. -15315.01 esetén -15315.009765625 a tárolt érték.

Online konverter: <https://www.h-schmidt.net/FloatConverter/IEEE754.html>

Egyszeres pontosság

- Nem normalizált számok

- Ha a *kitevőrés* = 0
- A *normalizálatlan* szám első jegye 0, így az nincs ábrázolva!
- Ábrázolt szám: $\pm(0 + \textit{törtrész}) \cdot 2^{\textit{kitevő}}$,
ahol *kitevő* = -126 rögzítetten!
 - Segítségével kezelhető az alulcsordulás: a *normalizátnál* kisebb értékeket reprezentálhatunk
 - Legkisebb *normalizált* szám: $1,0 \cdot 2^{-126}$
 - Legnagyobb *nem normalizált* szám: $0,99999999 \cdot 2^{-126}$
 - Legkisebb *nem normalizált* szám: $1,0 \cdot 2^{-149}$
 - A tizedesvesszőtől tovább jobbra tudjuk tolni a törtrészt, amit a normalizált forma nem engedett

Egyszeres pontosság

- Végtelen és érvénytelen szám (NaN) ábrázolása
 - Ha a *kitevőrész* = 255
 - Végtelen: csupa 0 törtrész bitminta
 - Érvénytelen szám: csupa 0-tól eltérő törtrész bitminta
 - NaN: *Not a Number*
 - Aritmetika
 - Szám / 0 = végtelen
 - Szám / végtelen = 0
 - Végtelen + bármi = végtelen
 - 0 / 0 = NaN
 - ...

Érdekesség

- **Fél-pontosságú lebegőpontos ábrázolás (binary16)**
 - Az IEEE 754-2008 szabványban bevezetve
 - Ábrázolás 16 biten: 1 bit előjel, 5 bites kitevő, 10 bites szignifikáns
 - Értéktartomány: $\pm 65,504$, az 1 értéknél nagyobb számok minimuma $1 + 1/1024$
 - Speciális bitminták
 - Exponens 00000_2 : nulla, –nulla, vagy normalizálatlan (szignifikáns értéke 0 vagy sem)
 - Exponens 00001_2 és 11110_2 között: normalizált érték
 - Exponens 11111 : $\pm$ végtelen vagy NaN (szignifikáns értéke 0 vagy sem)
 - **Előnyei, használata**
 - Jóval gyorsabb lehet ezzel számolni, mint a dupla pontossággal (erősen architektúra-függő!)
 - Kisebb tárhely és adatátviteli igény
 - Számítógépes grafika, képfeldolgozásban és mesterséges neuronhálók
 - A 16 bites egész típushoz képest nagyobb ábrázolási tartomány, ugyanannyi biten

Szöveg kódolása

- **Feladat**
 - Számértékekhez karakterek: szimbólumok, betűk, számok, írásjelek rendelése
- **Fontosabb kódolási rendszerek**
 - ASCII, bővített ASCII
 - ISO 8859-n kódlapok
 - Unicode (UTF-8, UTF-16, UTF-32)
- **Hogyan lesz a számértékből megjelenő betű a kijelzőn?**
 - Karakteres konzolon: minden kódszámhoz egy képmátrix (képernyő memóriába írva)
 - Grafikus kijelzőn: minden kódszámhoz egy vektorgrafikus leírás a megrajzolás módjáról (fontkészletek)

The image shows the lowercase letters 'a', 'b', and 'c' in a pixelated, monospaced font. Each letter is composed of a grid of black and white pixels, giving it a digital or retro appearance. The letters are positioned to the right of the text 'Hogyan lesz a számértékből megjelenő betű a kijelzőn?'.

ASCII (1963)

- A bájtok értékei karaktereket jelentenek
 - Latin ábécé betűi + írásjelek + vezérlőkódok
- Eredetileg az alsó 7 bit használatos
 - 0-31: vezérlő karakterek (pl. új sor, ESC, szöveg vége)
 - 32: szóköz
 - 33-47: írásjelek
 - 48-57: számjegyek
 - 58-64: nagybetűk
 - 91-96: írásjelek
 - 97-122: kisbetűk
 - 123-126: írásjelek
 - 127: DEL

ASCII Code Chart

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2		!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{		}	~	DEL

Ábra forrása: Wikipedia

Kiterjesztett ASCII (1963)

- 128-255 közötti értékek is definiáltak
 - pl. nemzeti karakterek, szimbólumok
- **Probléma**
 - Sok hiányzó karakter (pl. magyar Ő, Ű)

OEM Extended ASCII

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
8	ç	ü	é	â	ä	à	á	ç	ê	ë	è	ï	î	ì	ñ	ø
9	é	æ	æ	ô	ö	ò	û	ù	ÿ	ö	ü	ç	£	¥	℞	f
A	á	í	ó	ú	ñ	ñ	º	º	¿	¡	½	¾	¿	«	»	
B	⌘	⌘	⌘					n	q	h	h	h	h	u	u	u
C	⌘	⌘	⌘	⌘	⌘	⌘	⌘	⌘	⌘	⌘	⌘	⌘	⌘	⌘	⌘	⌘
D	⌘	⌘	⌘	⌘	⌘	⌘	⌘	⌘	⌘	⌘	⌘	⌘	⌘	⌘	⌘	⌘
E	α	β	Γ	Π	Σ	σ	μ	τ	ϑ	θ	Ω	δ	ω	ø	€	π
F	≡	±	≥	≤	ƒ	J	÷	≈	°	·	·	√	n	z	ı	

Ábra forrása: cplusplus.com

Cserélhető kódtáblák

- **ISO 8859 kódolási szabvány (1987)**
 - A különféle kiterjesztett karakterkészletek definiálására
 - ISO 8859-1, ... , ISO8859-16
 - ISO 8859-1: nyugat-európai karakterkészlet
 - ISO 8859-2: nyugat- és kelet-európai karakterkészlet (pl. magyar is)
 - ISO 8859-3: nyugat- és dél-európai karakterkészlet (pl. török, máltai + eszperantó)
 - ...
 - Egy dokumentumhoz egy kódtábla rendelhető: más kódtáblába kerülő nyelvek esetén probléma
 - Kínai, japán, karaktereket ebbe is lehetetlen belezsúfolni

ISO 8859-1

ISO/IEC 8859-1																
	x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	xA	xB	xC	xD	xE	xF
0x	^@	^A	^B	^C	^D	^E	^F	^G	^H	^I	^J	^K	^L	^M	^N	^O
1x	^P	^Q	^R	^S	^T	^U	^V	^W	^X	^Y	^Z	^[	^\	^]	^^	^_
2x	SP	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
3x	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4x	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5x	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6x	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7x	p	q	r	s	t	u	v	w	x	y	z	{		}	~	
8x	XXX	XXX	BPH	NBH	IND	NEL	SSA	ESA	HTS	HTJ	VTS	PLD	PLU	RI	SS2	SS3
9x	DCS	PU1	PU2	STS	CCH	MW	SPA	EPA	SOS	XXX	SCI	CSI	ST	OSC	PM	APC
Ax	NB SP	ı	ç	£	¤	¥	ı	§	¨	©	ª	«	¬	SHY	®	—
Bx	°	±	²	³	´	µ	¶	·	,	¹	º	»	¼	½	¾	¿
Cx	À	Á	Â	Ã	Ä	Å	Æ	Ç	È	É	Ê	Ë	Ì	Í	Î	Ï
Dx	Ð	Ñ	Ò	Ó	Ô	Õ	Ö	×	Ø	Ù	Ú	Û	Ü	Ý	Þ	ß
Ex	à	á	â	ã	ä	å	æ	ç	è	é	ê	ë	ì	í	î	ï
Fx	ð	ñ	ò	ó	ô	õ	ö	÷	ø	ù	ú	û	ü	ý	þ	ÿ

Forrás: https://hu.wikipedia.org/wiki/ISO/IEC_8859-1

Nyelvek

afrikaans, albán,
angol (UK és US),
baszk, breton,
holland, feröeri,
finn, galíciai, izlandi,
indonéz, ír, korzikai,
latin, leonese,
luxemburgi, maláj,
manx, német,
norvég, olasz,
okcitán, portugál,
romans, skót gael,
spanyol, szuahéli,
svéd, vallon

ISO 8859-2

ISO/IEC 8859-2																
	x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	xA	xB	xC	xD	xE	xF
0x	nem használt															
1x																
2x	SP	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
3x	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4x	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5x	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6x	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7x	p	q	r	s	t	u	v	w	x	y	z	{		}	~	
8x	nem használt															
9x																
Ax	NBSP	Ą	ˆ	Ł	ł	Ł	Ś	ś	˝	Š	š	Ť	Ž	SHY	Ž	Ž
Bx	°	ą	ˆ	ł	ł	ł	ś	ś	˝	š	š	ť	ž	˝	ž	ž
Cx	Ř	Á	Â	Ã	Ä	Å	Č	Ç	Ć	Ê	Ë	Ě	Í	Î	Ď	
Dx	Đ	Ń	Ň	Ó	Ô	Õ	Ö	×	Ř	Ů	Ú	Ů	Ü	Ý	Ť	ß
Ex	ř	á	â	ã	ä	å	č	ç	ć	ê	ë	ě	í	î	ď	
Fx	đ	ń	ň	ó	ô	õ	ö	÷	ř	ů	ú	ű	ü	ý	ť	·

Nyelvek

bosnyák, cseh,
horvát, lengyel,
magyar, román,
szerb (a latinbetűs
írasmóddal),
szerbhorvát,
szlovák, szlovén,
alsószorb és
felsőszorb

Alsó 128 kód:
változatlan

Felső 128 kód:
kódlaponként
változik, de
átfedések lehetnek.

UTF (Unicode) és ISO/IEC 10646 szabvány (1989-)

• Jellemzők

- Maximálisan 2,147,483,648 különböző karakter leírására
 - 128 csoport x 256 sík x 256 sor x 256 cella

• UTF-8

- Változó hosszúságú (elvileg 1-6, valójában 1-4 bájtos) tárolás
- Alsó hét bit: ASCII kódtábla
- További kódok
 - *Vezető bájtt* definiálja a *követő bájtok* számát (legfelső bitek 1 értékeivel)
 - A követő bájtok felső bitértékei rögzítettek (10)
 - U+10FFFF korlát (2003)
 - Max. 4 bájtos az UTF-8
 - 1,112,064 karakter
 - UTF-16-tal való kompatibilitás miatt
- Népszerű kódolás

Forrás: Wikipedia

Bits	Last code point	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6
7	U+007F	0xxxxxxx					
11	U+07FF	110xxxxx	10xxxxxx				
16	U+FFFF	1110xxxx	10xxxxxx	10xxxxxx			
21	U+1FFFFF	11110xxx	10xxxxxx	10xxxxxx	10xxxxxx		
26	U+3FFFFFF	111110xx	10xxxxxx	10xxxxxx	10xxxxxx	10xxxxxx	
31	U+7FFFFFFF	1111110x	10xxxxxx	10xxxxxx	10xxxxxx	10xxxxxx	10xxxxxx

Unicode

- UTF-16

- 16 vagy 32 bites (2 vagy 4 bájtos) tárolás
 - 0 és hexa 10FFFF közötti érvényes kódok
 - 1,112,064 karakter kódolására képes
- Bájtok sorrendje függ az architektúrától!
 - Kis- vagy nagy-endián
 - Opcionális BOM (bájt sorrend jelölő - *Byte order mark*) a szöveg elején
 - U+FEFF érték (0xEF,0xBB,0xBF bájtsorozat)
 - Ebből megállapítható a bájtsorrend
- Windows 2000 óta a Windows API ezt használja

- UTF-32

- Rögzített 32 bites (4 bájtos) tárolás
 - 0 és hexa 7FFFFFFF közötti érvényes kódok (2,147,483,648 lehetséges kód)
 - Bájtsorrend itt is problémás!
 - Szövegek helyett inkább csak 1-1 karakter leírására használják

Érdekességek



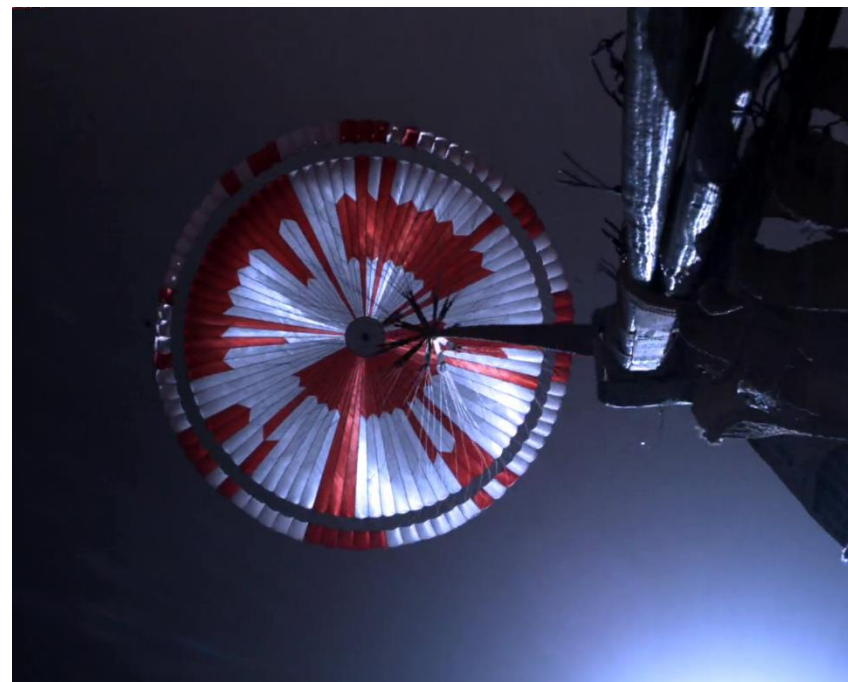
In the scene where RoboCop was being reprogrammed by Dr. Faxx, the following hex numbers scroll quickly up the screen: "50 45 54 45 20 4B 55 52 41 4E 20 49 53 20 41 20 47 52 45 41 54 20 47 55 59". Converted to ASCII text, it reads: "PETE KURAN IS A GREAT GUY". Peter Kuran was the special effects photographer.

- **RoboCop 2 mozifilm**

- RoboCop újraprogramozásakor egy hexa számsorozat fut végig
- A hexa számok ASCII karaktereket jelentenek

Érdekességek

- **Perseverance marsjáró ejtőernyőjén kódolt szöveg**
 - Megoldás magyarázat (angolul):
<https://www.nasa.gov/sites/default/files/atoms/files/mars-perseverance-parachute-coding-activity.pdf>
 - Wikipedia leírás (angolul):
[https://en.wikipedia.org/wiki/Perseverance_\(rover\)#Parachute_with_coded_message](https://en.wikipedia.org/wiki/Perseverance_(rover)#Parachute_with_coded_message)
- A kép közzététele után pár órán belül megfejtették...



Forrás: NASA/JPL-Caltech

Érdekességek



• ASCII art

- Vizuális információ ASCII karakterekből összeállítva
- Korábban írógépen, nyomtatott anyagban is!
- Vonalas alakzatok, árnyalás a betűk területkitöltésével, akár színek használatával

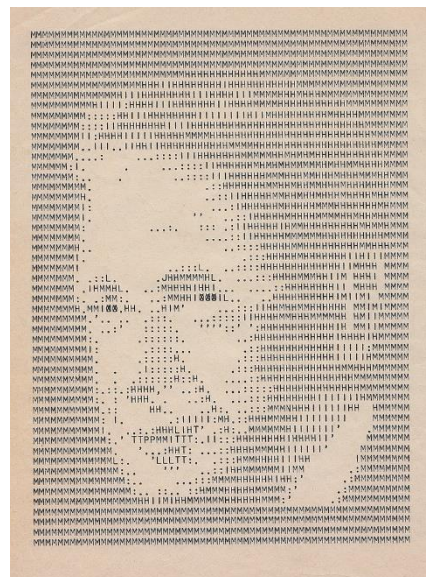
Front View



Rear View



The Character of Moa.
Made in JAPAN 2000
SVG Data converted by
MOTOI Kenkichi 2010.



Képek forrása: [ASCII Art](#) – a Wikipedia angol nyelvű bejegyzése

Bájtsorrend

- **Mostanra ismerjük**

- Központi memória rekeszes felépítését
 - Bájtos rekesz-felépítést feltételezünk a továbbiakban
- Számok bájtos reprezentációját

- **Kérdés**

- Ezek a reprezentációk hogyan jelennek meg a memóriában?

- **(Meglepő) válasz**

- **A több bájton reprezentált értékek memóriabeli megjelenése architektúra függő! Számít:**
 - Processzor regiszterszélessége (16, 32, vagy 64 bites)
 - Magas („*big*”) vagy alacsony („*little*”) helyiértékek bájtoi szerepelnek-e az alacsonyabb memóriacímeken
 - Nagy endián („*big endian*”) rendszerek: pl. SPARC, IBM nagygépek
 - Kis endián („*little endian*”) rendszerek: pl. Intel processzorok

Bájtsorrend

• Példa

```
typedef struct {
    int darabszam;
    short min;
    short max;
    float atlag;
    float szoras;
} statisztika;
```

...

```
statisztika stat1;
```

```
stat1.darabszam = 10;
stat1.min = -20;
stat1.max = 120;
stat1.atlag = 43.2;
stat1.szoras = 5.3;
```

Memóriatartalom

- 32 bites *kis endián* rendszeren
0a 00 00 00 ec ff 78 00 cd cc 2c 42 9a 99 a9 40
- 32 bites *nagy endián* rendszeren
00 00 00 0a ff ec 00 78 42 2c cc cd 40 a9 99 9a

Probléma

- Eltérő rendszerek közötti bináris adatátvitel nem kompatibilis!

Megoldási lehetőségek

- Szöveges kódolás fájlban tároláskor
 - (pl. XML, JSON)
- Bináris kódolás
 - 2 bájtos kód elhelyezése az adat elején, amiből az endián típus meghatározható (pl. „1” *short* értéke)
 - Ha szükséges, a bájtokat a memóriában fel kell cserélni beolvasás után

Hibadetektáló és -javító kódok

- **A memória hibázhat**

- A rekeszben tárolt érték nem programozottan megváltozik
- Pl. Áramlökés, hibás hardver hatására

- **Hatása**

- Adat területen: számítási hiba keletkezhet
- Kód területen: hibás kód keletkezhet, ha végrehajtódik, „lefagyhat” a program/rendszer

- **Kivédése**

- Redundancia bevezetésével
- *Detekció (felismerés)*
 - Bizonyos hibákat a rendszer „észrevesz”
 - Hibaüzenettel megállíthatja a program futását
- *Hibajavítás*
 - Bizonyos hibák esetén a legvalószínűbb helyes bájtérték helyreállítható

Érdekesség

- **A bitek átfordulása nem mindig véletlen**
 - <https://arstechnica.com/gadgets/2021/01/linus-torvalds-blames-intel-for-lack-of-ecc-ram-in-consumer-pcs/>
 - <https://arstechnica.com/information-technology/2019/06/researchers-use-rowhammer-bitflips-to-steal-2048-bit-crypto-key/>
 - <https://rambleed.com/>
 - Kutatók Rowhammer bitfordítással lopnak 2048 bites kriptokulcsot
 - **„Nem minden RAM-hiba hardverhiba vagy véletlen elektromágneses mező probléma eredménye.** Az elmúlt években a kutatók egyre praktikusabb fizikai alapú oldalcsatorna-támadásokat fejlesztettek ki, amelyek során a RAM egy alkalmazás számára hozzáférhető területein ellenőrzött, gyors bitfordításokkal következtetnek vagy módosítják a RAM szomszédos területein lévő adatok értékeit, amelyekhez nem lenne szabad hozzáférniük.”

Egyszerű példa: Paritásbit alkalmazása

- Minden bájtához plusz 1 bitet (paritásbit) hozzáveszünk
 - Így a memória 1/9-ed része nem tárol érdemi adatot
- A paritásbit a bájt értékben szereplő 1 értékek párosságát mutatja
 - Másként: biztosítja, hogy a 9 biten pl. mindig páros számú 1 érték legyen (lehetne páratlan is, ez megegyezés kérdése)
- Detekció
 - Ha a paritásbit értéke nem megfelelő, akkor memóriahiba fordul elő
- Példák
 - 00000000 **0**, 00001000 **1**, 11010011 **1** (érvényes kódok)
 - 000**1**0000 **0**, 000**1**1000 **1**, 11**1**00011 **0** (érvénytelen kódok)
 - 00**11**0000 **0**, 000**10**000 **1**, **00000111** **1** (érvényes kódok)
- Alkalmazhatósága
 - Csak páratlan számú bithibát (tehát nem mindent!)
 - tud detektálni (javításra nem képes)

Hibadetektáló és -javító kódok

• Definíciók

- m adat bit, r ellenőrző bit esetén $n = m + r$ bites a **kódszó**
- Hamming-távolság
 - Két bináris szám eltérő számjegyeinek száma
 - 10**00**101
 - 10**11**001 ezek Hamming-távolsága 3

• Detekciós és javító képesség

- 2^n kódszó közül csak 2^m darab érvényes
 - Összes érvényes kódszó listája: meghatározható az ellenőrző bitek számítási algoritmusának ismeretében
 - Összes érvényes kód Hamming-távolsága: az előző listában szereplő kódok közötti legközelebbi Hamming-távolság
- d egyszeres bithiba **felismeréséhez** $d + 1$ távolságú kódolás kell
- d egyszeres hiba **javításához** $2 \cdot d + 1$ távolságú kódolás kell
 - Hiba esetén: a legközelebbi Hamming-távolságú érvényeset vesszük

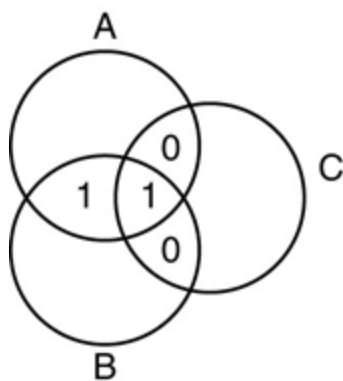
Hibadetektáló és -javító kódok

- **Kódolás egyszeres bithiba javítására**
 - m adat bit, r ellenőrző bit, $n = m + r$ bites kódszó
 - Alsó határ egyszeres bithiba javításához: $(m + r + 1) \leq 2^r$
 - *(Részleteket ld. a Tanenbaum könyvben)*
 - Nagyobb m érték esetén rövidebb kód, de időigényesebb ellenőrzés!

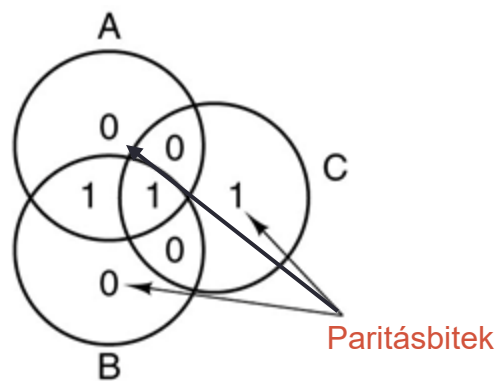
Szó hossza	Ellenőrző bitek	Teljes hossz	Hozzáadott bitek százaléka
8	4	12	50
16	5	21	31
32	6	38	19
64	7	71	11
128	8	136	6
256	9	265	4
512	10	522	2

Hibadetektáló és -javító kódok

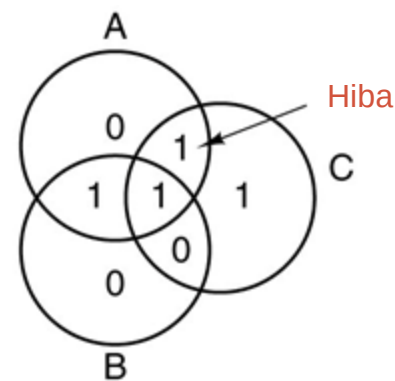
- Hamming-algoritmus 4 bites szóra



(a)



(b)



(c)

Forrás: Tanenbaum

- 1100 érték kódolása
- Paritásbitek számítása (A, B, C körökben páros számú 1 érték legyen)
- Egyszeres hiba megjelenése: A és C körökben elrontja a paritást. Javítás: ezen körök metszetében bit inverzió

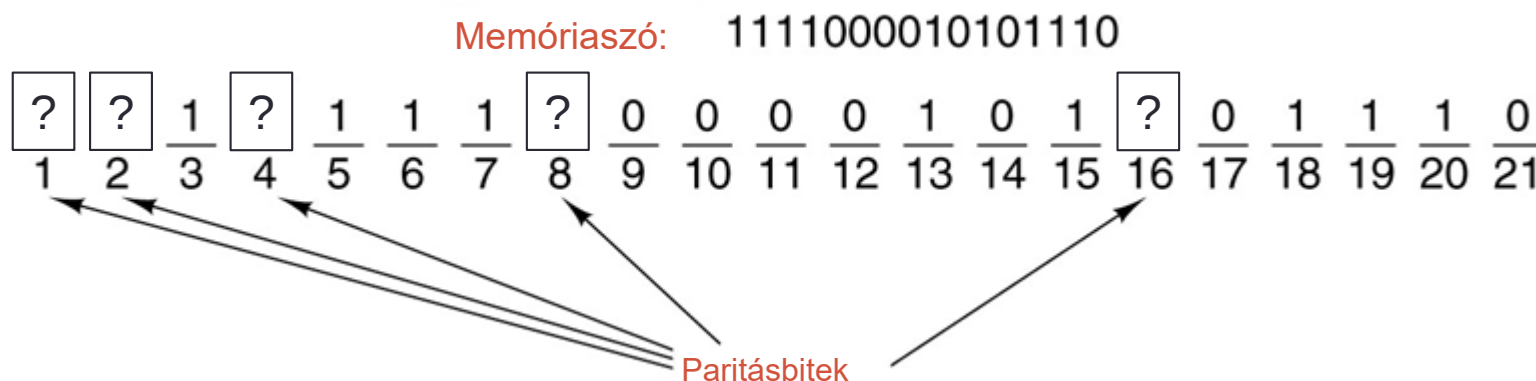
Hibadetektáló és -javító kódok

- **Hamming-algoritmus**

- m adat bit, r ellenőrző bit, $n = m + r$ bites kódszó
- Sorszámozási konvenció
 - Biteket 1-gyel kezdődően sorszámozzuk, a legnagyobb helyértékű az 1-es
- Paritásbitek
 - Amelyek sorszáma 2 egész kitevős hatványa (1, 2, 4, 8, 16, ...)
 - Minden paritásbit meghatározott bitpozíciókat ellenőriz
 - A b . bitet azok a $b_1, b_2, \dots, b_j$ paritásbitek ellenőrzik, amelyekre $b_1 + b_2 + \dots + b_j = b$
 - Értéke: biztosítja, hogy az ellenőrzött pozíciókon az 1-esek száma páros legyen
- Használata
 - Kiszámítjuk az össze paritásbitet, ha mindegyik helyes, akkor nincs javítható hiba
 - A hibás bit sorszámát a hibás paritásbitek sorszámainak összege adja
 - Ezt a bitet ellentettre állítjuk

Hibadetektáló és -javító kódok

- Hamming-algoritmus példa
 - 16 bites szó és 5 ellenőrző paritásbit



Hibadetektáló és -javító kódok

- Hamming-algoritmus példa
 - Paritásbit értékek kezdeti számítása

Memóriaszó: 1111000010101110

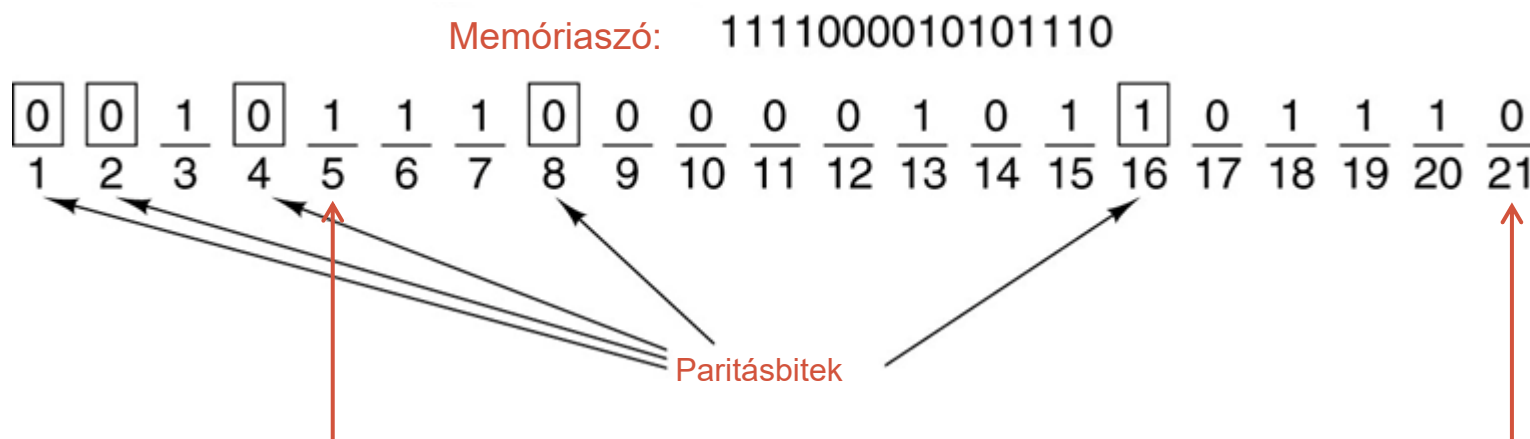


Paritásbitek

Memóriaszó	1	1	2	1	1	2	1	4	1	2	1	1	2	1	4	1					
bitekhez	2	4	4	2	8	8	2	8	4	4	2	16	16	2	16	4					
kapcsolódó				4			8		8	8	4			16		16					
paritásbitek											8										

Hibadetektáló és -javító kódok

- **Hamming-algoritmus példa**
 - 16 bites szó (+5 ellenőrző paritásbit)



- Hiba: tfh. az 5. bit átfordul, 0 lesz
 - Javítás: az 5. bithez az 1. és a 4. paritásbit tartozik (mivel $5 = 1 + 4$), ezek értéke hibás lesz. A többi helyes marad.
- Hiba: tfh. a 21. bit fordul át, 1 lesz
 - Javítás: az 1., 4. és 16. paritásbitek lesznek hibásak ($21 = 1 + 4 + 16$)

ECC és nem-ECC memóriák

- Megvásárolható PC RAM modulok
 - **Nem-ECC**
 - Nincs hibaellenőrzés
 - Sem paritás, sem korrekció!
 - Olcsóbb
 - A mai memóriamodulok már ritkán hibáznak, ezért ez a gyakoribb
 - De van más vélemény is... (lásd egy későbbi dián)
 - **ECC (error correcting code)**
 - Hibadetektálás és javítás
 - 1 bithiba javítása és 2 bithiba detektálása 64 bites szavanként
 - Szerverekbe és kritikus alkalmazásokat kiszolgáló számítógépekbe
 - Alaplap támogatás szükséges

www.arukereso.hu



[HP 8GB \(2x4GB\) DDR2 667MHz 466440-B21](#)

667 MHz, 8 GB, DDR2

HP (466440-B21) HP 8GB Fully Buffered DIMM PC2-5300 2x4GB Low Power DDR2 Memory Kit Így is ismerheted: 8 GB 2 x 4 GB DDR 2 667 MHz 466440 B 21, 8GB2x4GBDDR2667MHz466440B21, 8GB (2x4GB)...

138 003 Ft

[irány a bolt >](#)

[senetic.hu](#)

[Összehasonlítás](#)



[Kingston 32GB \(4x8GB\) DDR3 1600MHz D1G72K111K4](#)

1600 MHz, 32 GB, DDR3

KIT kiszerelésben, 4 darabos csomagban kaphatóakGyors, 32GB kapacitású memóriamodul. ECC regisztrált, integrált hőelvezető. Kifejezetten Szerver alaplaphoz ajánlott. DDR3-as technológia...

134 300 Ft

[irány a bolt >](#)

[bluechip.hu](#)

[Összehasonlítás](#)



[Corsair 32GB \(4x8GB\) DDR3 2133MHz CMZ32GX3M4A2133C10](#)

2133 MHz, 32 GB, DDR3

Kit kiszerelés, azaz 4db memóriamodul!Dual Channel: 4 csatornás memóriabusz, a lehető leggyorsabb adatelérés érdekében!A Corsair Vengeance DDR3 memóriamodulokat a tuningolókat szem...

130 000 Ft

[irány a bolt >](#)

[bluechip.hu](#)

[Összehasonlítás](#)



[Fujitsu 16GB DDR3 1066MHz S26361-F3604-L516](#)

1066 MHz, 16 GB, DDR3

Fujitsu S26361-F3604-L516 RAM Module - 16 GB - DDR3 SDRAM - 1066 MHz DDR3-1066/PC3-8500 - Registered Így is ismerheted: 16 GB DDR 3 1066 MHz S 26361 F 3604 L 516, 16GBDDR31066MHzS26361F3604L516,...

127 688 Ft

[irány a bolt >](#)

[webshop.a4team.hu](#)

[Összehasonlítás](#)

KIT kiszerelésben, 4 darabos csomagban kaphatóakGyors, 32GB kapacitású memóriamodul. ECC regisztrált, integrált hőelvezető. Kifejezetten Szerver alaplaphoz ajánlott. DDR3-as technológia. 1600MHz frekvencia sebességgel rendelkezik. Minden Kingston szerver memória a legújabb vezető gyártási technológia eredménye. Garancia a minőségre. Stabilitás, gyorsaság, 100%-os kompatibilitás jellemzi. A Kingston a lehető legjobb forrás, ha rendszer specifikus szerver memóriát...

[Kingston 32GB \(4x8GB\) DDR3 1600MHz D1G72K111K4 teljes leírás >](#)

Why don't PCs use error correcting RAM?

“Because Intel,” says Linus

- **Linus Torvalds véleménye az ECC RAM-ok elterjedése kapcsán**
 - <https://arstechnica.com/gadgets/2021/01/linus-torvalds-blames-intel-for-lack-of-ecc-ram-in-consumer-pcs/> (Ars Technica, Jim Salter - 1/6/2021)
 - „Torvalds azt a merész álláspontot képviseli, hogy az ECC RAM hiánya a fogyasztói technológiákban az Intel hibája, ami a vállalat mesterséges piacszegmentációs politikájának köszönhető. Az Intelnek érdeke, hogy a mélyebb pénztárcájú vállalkozásokat a drágább – és jövedelmezőbb – server CPU-k felé terelje, ahelyett, hogy hagyná, hogy ezek a szervezetek hatékonyan használják a szükségszerűen alacsonyabb árrésű fogyasztói alkatrészeket.”
 - ...
 - „A szokásos érv, amiért az ECC nincs jelen a fogyasztói technológiában, a költségek körül forog, de gyanítjuk, hogy Torvaldsnak itt igaza van. Annak ellenére, hogy az ECC RAM lényegében egy nehezen beszerezhető speciális alkatrész, jellemzően csak körülbelül 20 százalékkal kerül többbe DIMM-enként, mint a nem ECC-sek a kiskereskedelemben. Az igazi probléma az, hogy az ezt támogató alaplapok és processzorok nélkül ez a megoldás nem sokat ér.”

Gyakorló feladatok

- **Váltsa át az alábbi decimális értékeket bináris, oktális és hexadecimális számrendszerekbe!**
 - 34
 - 109
 - 243
 - 732
 - 39513
- **Váltsa át az alábbi decimális törtszámokat bináris formába!**
 - 0.5
 - 0.45
 - -4.678

Gyakorló feladatok

- **Az alábbiak közül melyek érvényes számok az adott számrendszerekben?**
 - 101_2 , 0_2 , 12_2
 - 0_8 , 77_8 , 19_8
 - F_{16} , $988A_{16}$, $GAGA_{16}$, $DEAD_{16}$, BAD_{16} , $CAFE_{16}$
- **Végezze el az alábbi átváltásokat!**
 - $01101011_2 \rightarrow$ nyolcas és tizenhatos számrendszerbe
 - $CAB_{16} \rightarrow$ kettes és nyolcas számrendszerbe

Gyakorló feladatok

- **Átszámolás decimálisból bitmintába**

- Gondoljon egy számra -32768 és +32767 között! Alakítsa át az alábbi bináris egész reprezentációkba:
 - Előjel nélküli, előjeles abszolútérték, egyes és kettes komplement, 16383 többletes
 - A bináris értéket adja meg hexadecimális számként is!

- **Átszámolás bitmintából decimálisba**

- A 101011011 bitminta milyen decimális értéket reprezentál
 - előjel nélküli,
 - előjeles abszolútérték,
 - egyes és kettes komplement,
 - 127 többletes formában?
- Írjon fel egy véletlenszerű 8 illetve 16 számból álló 0-1 számjegy sorozatot
 - Végezze el az előző feladat számításait!

Gyakorló feladatok

- **Tegyük fel, hogy egy decimális, előjel nélküli egész számértéket tartalmazó szövegfájl ASCII formátumban beolvassunk a központi memóriába. A központi memória ehhez kapcsolódó memóriaterületének tartalma hexadecimális formátumban 33 31 35 32 30 34.**
 - Adjunk algoritmust a bájtsorozatból decimális számérték meghatározására!
 - Hogyan módosulna az algoritmus, ha opcionálisan meglévő előjelet is figyelembe szeretnénk venni?
 - Mit kellene tenni, hogy decimális törtértékeket is át tudjunk számolni? (Feltesszük, hogy tizedespontot használunk elválasztó karakternek.)
 - Mit csináljunk, ha érvényes számjegy ellenőrzést is szeretnénk végezni?

Gyakorló feladatok

- A központi memória egy adott celláiban a **4E 65 75 6D 61 6E 6E 00** számsorozatot találjuk. Mit reprezentálhatnak ezek a bájtok?
 - Ha 2 vagy 4 bájtos egész számokat jelentenek?
 - Ha **float** típusú számokat? És ha **double**-t?
 - Ha szöveget ASCII formátumban? És ha UTF-8-ban?
 - Azt megkérdezte az első kérdésnél, hogy kis- vagy nagy-endián rendszerről van szó? 😊

Gyakorló feladatok

- **Hamming kódok távolsága**
 - Írjon fel két egyforma hosszúságú számsorozatot a 0 és 1 számjegyekből. Számítsa ki ezek Hamming kód távolságát!

Gyakorló feladatok

- **Hamming algoritmus 4-bitre**

- Tegyük fel, hogy a memóriában tárolt 4 adatbit a következő értékű: $m_1=1$, $m_2=0$, $m_3=0$, $m_4=1$. A paritásbitek páros paritást vizsgálnak.
- Mik lesznek a p_1 , p_2 , p_3 paritásbit kezdeti értékek?
- Hogyan működik a javító algoritmus, ha az m_2 adatbit értéke nem programozottan megváltozik?
- Mi történik, ha memóriahiba a p_1 paritásbitben jelentkezik?
- Mi történik, ha az m_3 és m_4 egyszerre változik meg?
- Változtasson az $m_1, \dots, m_4$ kezdeti értékeken, és válaszolja meg így is a kérdéseket!

